



# Relatório do Projeto de Estruturas de Dados Avançados

Tiago Rodrigo Soares Afonso

Nº 20452 – Regime Diurno

Professor

Luís Gonzaga Martins Ferreira

Ano letivo 2024/2025

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

## ÍNDICE

|        |                                         |       |
|--------|-----------------------------------------|-------|
| 1.     | Introdução .....                        | V     |
| 1.1.   | Objetivos.....                          | VI    |
| 1.2.   | Fase 1 – Listas Ligadas .....           | VI    |
| 1.3.   | Fase 2 – Grafos .....                   | VI    |
| 2.     | Fase 1 – Listas Ligadas .....           | VIII  |
| 2.1.   | Definição da Estrutura de Dados .....   | VIII  |
| 2.2.   | Antena .....                            | X     |
| 2.2.1. | Cabeçalho (Antena.h) .....              | X     |
| 2.2.2. | Implementação (Antena.cpp) .....        | XI    |
| 2.3.   | Lista Antenas .....                     | XIV   |
| 2.3.1. | Cabeçalho (ListaAntenas.h) .....        | XIV   |
| 2.3.2. | Implementação (ListaAntenas.cpp) .....  | XIV   |
| 2.4.   | Local Nefasto .....                     | XVIII |
| 2.4.1. | Cabeçalho (LocalNefasto.h) .....        | XVIII |
| 2.4.2. | Implementação (LocalNefasto.cpp): ..... | XVIII |
| 2.5.   | Projeto_EDA.cpp (main.c) .....          | XXII  |
| 2.5.1. | exibirMenu (Menu Interativo).....       | XXII  |
| 2.5.2. | main (Função principal).....            | XXII  |
| 2.6.   | Testes ao programa.....                 | XXIV  |
| 2.6.1. | Menu .....                              | XXIV  |
| 2.6.2. | Listar Antenas .....                    | XXIV  |
| 2.6.3. | Listar Antenas Formatado .....          | XXIV  |
| 2.6.4. | Inserir Antena .....                    | XXV   |
| 2.6.5. | Remover Antena .....                    | XXV   |
| 2.6.6. | Listar Locais Nefastos .....            | XXVI  |

|                                               |       |
|-----------------------------------------------|-------|
| 2.6.7. Listar Locais Nefastos Formatado ..... | XXVI  |
| 3. Conclusão .....                            | XXVII |

## ÍNDICE DE FIGURAS

|                                                            |       |
|------------------------------------------------------------|-------|
| Figura 1 - Estruturas de Dados .....                       | IX    |
| Figura 2 - Antena.h.....                                   | X     |
| Figura 3 - Verifica posição Antena .....                   | XI    |
| Figura 4 - Insere Antena.....                              | XII   |
| Figura 5 - Remover Antena .....                            | XII   |
| Figura 6 - Inserir e remover Antenas Manual .....          | XIII  |
| Figura 7 - Lista de Antenas (h).....                       | XIV   |
| Figura 8 - Criação da Estrutura de Dados das Antenas ..... | XIV   |
| Figura 9 - Destrói a Estrutura de Dados.....               | XV    |
| Figura 10 - Carrega Antenas do Ficheiro .....              | XV    |
| Figura 11 - Lista Antenas .....                            | XVI   |
| Figura 12 - Lista Antenas de Forma Formatada.....          | XVII  |
| Figura 13 - Cabeçalho LocalNefasto(.h).....                | XVIII |
| Figura 14 - Local Nefasto Existe .....                     | XVIII |
| Figura 15 - Adiciona Local Nefasto .....                   | XIX   |
| Figura 16 - Calcula Locais Nefastos.....                   | XX    |
| Figura 17 - Listar Locais Nefastos.....                    | XXI   |
| Figura 18 - Lista Locais Nefastos Formatado.....           | XXI   |
| Figura 19 - Menu Interativo .....                          | XXII  |
| Figura 20 - Função principal (Main).....                   | XXIII |
| Figura 21 - Menu .....                                     | XXIV  |
| Figura 22 - Lista de Antenas Consola .....                 | XXIV  |
| Figura 23 - Lista Antenas Formatado Consola.....           | XXIV  |
| Figura 24 - Inserir Antena Consola.....                    | XXV   |
| Figura 25 - Remover Antena Consola.....                    | XXV   |
| Figura 26 - Lista Locais Nefastos Consola.....             | XXVI  |
| Figura 27 - Lista Locais Nefastos Formatada Consola.....   | XXVI  |

# 1. Introdução

Este projeto destina-se à avaliação dos conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas (EDA) e foca-se na implementação e manipulação de estruturas de dados dinâmicas, bem como na aplicação de conceitos avançados de teoria dos grafos. O seu desenvolvimento, realizado em linguagem C, propõe a resolução de um problema de dimensão média através da definição de estruturas de dados específicas para a representação de antenas e respetivos efeitos nefastos, decorrentes de interações baseadas em frequências de ressonância.

A primeira fase do projeto centra-se na utilização de listas ligadas para a gestão de dados referentes às antenas e às localizações afetadas, abrangendo operações de inserção, remoção e dedução automática dos pontos críticos. Na segunda fase, o desafio é reinterpretado através de uma abordagem baseada em grafos, onde cada vértice representa uma antena e as arestas ligam antenas com frequências iguais, permitindo a exploração do espaço através de buscas em profundidade e largura, e a identificação de caminhos e intersecções entre pares de antenas de frequências distintas.

Esta iniciativa não só reforça a compreensão dos conceitos teóricos, mas também promove a sua aplicação prática através de um desenvolvimento modular e rigorosamente documentado.

O projeto constitui, assim, uma oportunidade de integrar e aplicar conhecimentos de estruturas de dados avançadas, contribuindo para a formação de competências essenciais à resolução de problemas computacionais complexos.

## 1.1. Objetivos

- Consolidar os conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas (EDA) através da aplicação prática em linguagem C.
- Implementar estruturas de dados dinâmicas para a representação de antenas, utilizando listas ligadas na Fase 1 e grafos na Fase 2.
- Desenvolver um mecanismo para carregar e interpretar os dados de antenas a partir de um ficheiro de texto, onde as antenas são representadas numa matriz de caracteres de dimensões variáveis.

## 1.2. Fase 1 – Listas Ligadas

- ✓ Definir e manipular uma estrutura de dados (lista ligada) para armazenar a frequência de ressonância e as coordenadas das antenas.
- ✓ Implementar operações de inserção e remoção de antenas na lista ligada.
- ✓ Deduzir automaticamente as localizações com efeito nefasto, conforme a regra de alinhamento e distância entre pares de antenas com a mesma frequência, e representá-las através de uma lista ligada.
- ✓ Exibir, de forma tabular na consola, tanto os dados das antenas como as localizações identificadas com efeito nefasto.

## 1.3. Fase 2 – Grafos

- Definir um tipo de dados para a representação de grafos (GR), onde cada vértice representa uma antena e as arestas conectam antenas com frequências de ressonância iguais.
- Carregar os dados das antenas de um ficheiro de texto para construir o grafo correspondente.
- Implementar operações de manipulação de grafos, que incluam:
  - Procura em profundidade (DFS) a partir de uma antena, listando as coordenadas das antenas alcançadas.
  - Procura em largura (BFS) a partir de uma antena, com listagem semelhante.

- Identificação e listagem de todos os caminhos possíveis entre duas antenas, apresentando as sequências de arestas interligadas.
- Listagem de intersecções de pares de antenas com frequências distintas, indicando as respectivas coordenadas.

## 2. Fase 1 – Listas Ligadas

A Fase 1 do projeto centrou-se na implementação de estruturas de dados dinâmicas através da utilização de listas ligadas para representar as antenas numa cidade. Cada antena foi identificada pela sua frequência de ressonância e coordenadas espaciais, armazenadas num registo da lista ligada. O objetivo foi desenvolver operações essenciais como inserção, remoção e dedução automática das localizações com efeito nefasto, com base na disposição e frequência das antenas. Além disso, será realizada uma listagem tabular na consola para visualizar tanto as antenas registadas como as localizações críticas identificadas.

### 2.1. Definição da Estrutura de Dados

Foram definidas estruturas de dados para representar antenas e locais nefastos utilizando listas ligadas em C no ficheiro **Dados.h**.

Este ficheiro tem os seguintes dados:

- **Antena:** Estrutura com a frequência (caracter), coordenadas (linha e coluna) e ponteiro para a próxima antena.
- **ED:** Estrutura principal que contém o ponteiro para a cabeça da lista de antenas.
- **LocalNefasto:** Estrutura com coordenadas (linha e coluna) e ponteiro para o próximo local nefasto.
- **ED\_LocaisNefastos:** Estrutura principal que contém o ponteiro para a cabeça da lista de locais nefastos.

Pode observar-se a estrutura na **Figura 1**.



```

#ifndef DADOS_H
#define DADOS_H

/**
 * @brief Estrutura que representa uma antena na lista ligada
 */
typedef struct Antena {
    char frequencia;    ///< Frequência da antena (ex: 'A', 'O')
    int linha;          ///< Linha na matriz (coordenada x)
    int coluna;         ///< Coluna na matriz (coordenada y)
    struct Antena* prox; ///< Ponteiro para a próxima antena na lista
} Antena;

/**
 * @brief Estrutura principal ED (lista ligada de antenas)
 */
typedef struct {
    Antena* cabeca;    ///< Ponteiro para a primeira antena da lista
} ED;

/**
 * @brief Estrutura que representa um local nefasto na lista ligada
 */
typedef struct LocalNefasto {
    int linha;
    int coluna;
    struct LocalNefasto* prox;
} LocalNefasto;

/**
 * @brief Estrutura principal LocaisNefastos (lista ligada de locais nefastos)
 */
typedef struct {
    LocalNefasto* cabeca; // Lista de locais nefastos
} ED_LocaisNefastos;

#endif // DADOS_H

```

Figura 1 - Estruturas de Dados

## 2.2. Antena

### 2.2.1. Cabeçalho (Antena.h)

Nesta classe foram declaradas as funções para verificar a posição da antena, inserir e remover antenas, e manipular manualmente os dados das antenas.

Aqui os dados só estão declarados e não implementados.

```
#pragma once
#ifndef ANTENA_H
#define ANTENA_H

#include "Dados.h" // Para usar as estruturas Antena e ED

// ----- Operações Básicas -----
int verificaPosicaoAntena(ED* ed, int linha, int coluna);
void insereAntena(ED* ed, char freq, int linha, int coluna);
int removerAntena(ED* ed, char freq, int linha, int coluna);
void inserirAntenaManual(ED* ed);
void removerAntenaManual(ED* ed);

#endif // ANTENA_H
```

*Figura 2 - Antena.h*

Pode observar-se a declaração dos métodos sem a implementação na **Figura**

2.

### 2.2.2. Implementação (Antena.cpp)

Nesta classe foram implementados os métodos declarados anteriormente como:

**verificaPosicaoAntena:** Confirma se já existe uma antena numa posição específica percorrendo a lista ligada e verificando se alguma delas já está na posição específica (linha, coluna). **(Figura 3).**

```
int verificaPosicaoAntena(ED* ed, int linha, int coluna) {  
    // Ponteiro auxiliar para percorrer a lista de antenas  
    Antena* atual = ed->cabeca;  
  
    // Percorre a lista ligada até encontrar uma correspondência ou chegar ao final  
    while (atual != NULL) {  
        // Verifica se a antena atual está na mesma posição especificada  
        if (atual->linha == linha && atual->coluna == coluna) {  
            return 0; // Retorna 0 indicando que a posição já está ocupada  
        }  
        atual = atual->prox; // Avança para a próxima antena na lista  
    }  
  
    return 1; // Retorna 1 indicando que a posição está disponível  
}
```

*Figura 3 - Verifica posição Antena*

**insereAntena:** Insere uma nova antena no início da lista ligada, após verificar a disponibilidade da posição. Este método tem como argumentos de entrada a Lista Ligada, a frequência, a linha e a coluna da nova antena. **(Figura 4).**

```

void insereAntena(ED* ed, char freq, int linha, int coluna) {

    // Verifica se a posição já está ocupada por outra antena
    if (!verificaPosicaoAntena(ed, linha, coluna)) {
        printf("Erro: Já existe uma antena em (%d, %d).\n", linha, coluna);
        return; // Sai da função sem inserir a nova antena
    }

    // Aloca memória para a nova antena
    Antena* novaAntena = (Antena*)malloc(sizeof(Antena));
    if (novaAntena == NULL) {
        perror("Erro ao alocar memória para Antena");
        exit(EXIT_FAILURE); // Termina o programa em caso de erro de alocação
    }

    // Inicializa os valores da nova antena
    novaAntena->frequencia = freq;
    novaAntena->linha = linha;
    novaAntena->coluna = coluna;

    // Insere a nova antena no início da lista ligada
    novaAntena->prox = ed->cabeca;
    ed->cabeca = novaAntena;
}

```

*Figura 4 - Insere Antena*

**removerAntena:** Remove a antena que corresponde à frequência e coordenadas fornecidas caso ela realmente esteja presente na lista ligada, atualizando a lista ligada seguidamente. **(Figura 5)**

```

int removerAntena(ED* ed, char freq, int linha, int coluna) {

    Antena* atual = ed->cabeca; // Ponteiro para percorrer a lista
    Antena* anterior = NULL;    // Ponteiro para manter referência ao nó anterior

    // Percorre a lista até encontrar a antena ou chegar ao final
    while (atual != NULL) {

        // Verifica se a antena atual corresponde aos critérios de remoção
        if (atual->linha == linha && atual->coluna == coluna && atual->frequencia == freq) {

            // Se a antena a ser removida for a primeira da lista
            if (anterior == NULL) {
                ed->cabeca = atual->prox; // Atualiza a cabeça da lista
            }
            else {
                anterior->prox = atual->prox; // Salta o nó a ser removido
            }

            free(atual); // Liberta a memória da antena removida
            return 1; // Retorna 1 indicando remoção bem-sucedida
        }

        // Avança para o próximo nó, mantendo o anterior registrado
        anterior = atual;
        atual = atual->prox;
    }

    return 0; // Retorna 0 caso a antena não tenha sido encontrada na lista
}

```

*Figura 5 - Remover Antena*

**inserirAntenaManual/removerAntenaManual:** Permite ao utilizador introduzir os dados da antena que deseja inserir ou remover pela consola. Assim estas funções guardam os argumentos inseridos e são utilizados nos métodos de inserir e remover criados anteriormente. **(Figura 6)**

```
// Função para inserir uma antena manualmente
void inserirAntenaManual(ED* ed) {
    char freq;
    int linha, coluna;

    // Ler frequência
    printf("Insira a frequência da antena (caractere) a acrescentar: ");
    scanf_s(" %c", &freq, 1);

    // Ler coordenadas
    printf("Insira a linha da antena: ");
    scanf_s("%d", &linha);

    printf("Insira a coluna da antena: ");
    scanf_s("%d", &coluna);

    // Chamar função de inserção
    insereAntena(ed, freq, linha, coluna);
}

// Função para remover uma antena manualmente
void removerAntenaManual(ED* ed) {
    char freq;
    int linha, coluna;

    // Ler frequência
    printf("Insira a frequência da antena (caractere) a remover: ");
    scanf_s(" %c", &freq, 1);

    // Ler coordenadas
    printf("Insira a linha da antena: ");
    scanf_s("%d", &linha);

    printf("Insira a coluna da antena: ");
    scanf_s("%d", &coluna);

    // Chamar função de remoção
    removerAntena(ed, freq, linha, coluna);
}
```

*Figura 6 - Inserir e remover Antenas Manual*

## 2.3. Lista Antenas

### 2.3.1. Cabeçalho (ListaAntenas.h)

Neste ficheiro foram declaradas funções para criar e destruir a estrutura de dados (ED), carregar antenas a partir de ficheiros de texto, e listar as antenas de diferentes formas. **(Figura 7)**

```
#pragma once

#ifndef LISTAANTENAS_H
#define LISTAANTENSA_H

#include "Dados.h" // Para usar as estruturas Antena e ED

// ----- Gestão da ED -----
ED* criarED();
void destruirED(ED* ed);
void carregarAntenasDeFicheiro(ED* ed, const char* nomeFicheiro, int* max_linhas, int* max_colunas);
void listarAntenas(const ED* ed);
void listarAntenasFormatado(const ED* ed, int max_linhas, int max_colunas);

#endif // ANTENA_H
```

*Figura 7 - Lista de Antenas (h)*

### 2.3.2. Implementação (ListaAntenas.cpp)

Nesta classe foram implementados os métodos declarados anteriormente como:

**criarED:** Aloca memória para a estrutura e inicializa a lista como vazia. **(Figura 8)**

```
ED* criarED() {
    // Aloca memória para a estrutura ED
    ED* ed = (ED*)malloc(sizeof(ED));
    if (ed == NULL) {
        perror("Erro ao alocar memória para ED");
        exit(EXIT_FAILURE); // Encerra o programa em caso de erro na alocação
    }

    ed->cabeca = NULL; // Inicializa a lista como vazia
    return ed; // Retorna o ponteiro para a estrutura criada
}
```

*Figura 8 - Criação da Estrutura de Dados das Antenas*

**destruirED:** Liberta toda a memória alocada para a lista ligada, removendo cada antena. **(Figura9).**

```

void destruirED(ED* ed) {
    Antena* atual = ed->cabeca; // Ponteiro para percorrer a lista

    // Percorre a lista, liberando a memória de cada antena
    while (atual != NULL) {
        Antena* prox = atual->prox; // Guarda referência para o próximo nó
        free(atual); // Liberta a memória da antena atual
        atual = prox; // Avança para o próximo nó
    }

    free(ed); // Liberta a memória da estrutura ED
}

```

Figura 9 - Destrói a Estrutura de Dados

**carregarAntenasDeFicheiro:** Lê um ficheiro de texto contendo uma matriz de caracteres. Cada célula pode conter uma antena ou um caractere "." indicando espaço vazio. Insere antenas na lista conforme encontrado no ficheiro. (Figura 10)

```

void carregarAntenasDeFicheiro(ED* ed, const char* nomeFicheiro, int* max_linhas, int* max_colunas) {
    FILE* arquivo = NULL;

    // Abre o ficheiro para leitura
    errno_t err = fopen_s(&arquivo, nomeFicheiro, "r");
    if (err != 0 || arquivo == NULL) {
        perror("Erro ao abrir o arquivo");
        return;
    }

    char buffer[1024]; // Buffer para armazenar cada linha lida do ficheiro
    int linhaAtual = 0;
    *max_colunas = 0;

    // Lê o ficheiro linha por linha
    while (fgets(buffer, sizeof(buffer), arquivo) != NULL) {
        size_t comprimento = strlen(buffer);

        // Remove o caractere de nova linha ('\n') se presente
        if (comprimento > 0 && buffer[comprimento - 1] == '\n') {
            buffer[comprimento - 1] = '\0';
            comprimento--;
        }

        // Atualiza o número máximo de colunas encontrado
        if (comprimento > *max_colunas) {
            *max_colunas = comprimento;
        }

        // Processa cada caractere da linha
        for (int coluna = 0; coluna < comprimento; coluna++) {
            if (buffer[coluna] != '.') { // Se não for '.', é uma antena
                insereAntena(ed, buffer[coluna], linhaAtual, coluna);
            }
        }

        linhaAtual++; // Avança para a próxima linha
    }

    *max_linhas = linhaAtual; // Atualiza o número total de linhas lidas
    fclose(arquivo); // Fecha o ficheiro após a leitura
}

```

Figura 10 - Carrega Antenas do Ficheiro

### Listagem de antenas ordenadas:

Conta o número de antenas na lista. Armazena os ponteiros num array e ordena-os por linha (decrecente) e coluna (crescente). Exibe as antenas ordenadas na consola. (Figura 11)

```
void listarAntenas(const ED* ed) {
    // Passo 1: Contar o número total de antenas na lista
    int numAntenas = 0;
    Antena* atual = ed->cabeca;
    while (atual != NULL) {
        numAntenas++;
        atual = atual->prox;
    }

    // Passo 2: Alocar um array de ponteiros para armazenar as antenas
    Antena** arrayAntenas = (Antena**)malloc(numAntenas * sizeof(Antena*));
    if (arrayAntenas == NULL) {
        perror("Erro ao alocar memória");
        return;
    }

    // Passo 3: Preencher o array com as antenas da lista ligada
    atual = ed->cabeca;
    for (int i = 0; i < numAntenas; i++) {
        arrayAntenas[i] = atual;
        atual = atual->prox;
    }

    // Passo 4: Ordenar as antenas primeiro por linha (decrecente), depois por coluna (crescente)
    qsort(arrayAntenas, numAntenas, sizeof(Antena*),
    [](const void* a, const void* b) -> int {
        Antena* antenaA = *(Antena**)a;
        Antena* antenaB = *(Antena**)b;

        // Ordenação decrecente por linha
        if (antenaA->linha != antenaB->linha) {
            return antenaB->linha - antenaA->linha; // Inverter a ordem
        }

        // Se as linhas forem iguais, ordenar por coluna de forma crescente
        return antenaA->coluna - antenaB->coluna;
    });

    // Passo 5: Exibir a lista de antenas ordenadas
    printf("=== Lista de Antenas (Ordenadas) ===\n");
    for (int i = 0; i < numAntenas; i++) {
        printf("Antena '%c' em (%d, %d)\n",
            arrayAntenas[i]->frequencia,
            arrayAntenas[i]->linha,
            arrayAntenas[i]->coluna);
    }
    printf("=====\n");

    // Passo 6: Libertar a memória alocada para o array de ponteiros
    free(arrayAntenas);
}
```

Figura 11 - Lista Antenas



### Listagem formatada de antenas (formato de Ficheiro de Entrada):

Cria uma matriz que representa a disposição das antenas. Preenche a matriz com caracteres “.” e substitui pelas frequências das antenas conforme a lista. Imprime a matriz de forma visualmente organizada. **(Figura 12)**

```
void listarAntenasFormatado(const ED* ed, int max_linhas, int max_colunas) {
    if (ed == NULL || ed->cabeca == NULL) {
        printf("Nenhuma antena encontrada.\n");
        return;
    }

    // Alocar dinamicamente uma matriz de caracteres (array de strings)
    char** matriz = (char**)malloc(max_linhas * sizeof(char*));
    if (matriz == NULL) {
        perror("Erro ao alocar memória para a matriz");
        exit(EXIT_FAILURE);
    }

    // Alocar e inicializar cada linha da matriz
    for (int i = 0; i < max_linhas; i++) {
        matriz[i] = (char*)malloc((max_colunas + 1) * sizeof(char)); // +1 para '\0'
        if (matriz[i] == NULL) {
            perror("Erro ao alocar memória para uma linha da matriz");
            exit(EXIT_FAILURE);
        }
        // Preencher a linha com '.' e adicionar o terminador de string '\0'
        for (int j = 0; j < max_colunas; j++) {
            matriz[i][j] = '.';
        }
        matriz[i][max_colunas] = '\0';
    }

    // Percorrer a lista de antenas e marcar suas posições na matriz
    Antena* atual = ed->cabeca;
    while (atual != NULL) {
        if (atual->linha >= 0 && atual->linha < max_linhas &&
            atual->coluna >= 0 && atual->coluna < max_colunas) {
            matriz[atual->linha][atual->coluna] = atual->frequencia; // Marca a antena na matriz
        }
        atual = atual->prox;
    }

    // Imprimir a matriz formatada na consola
    printf("=== Mapa de Antenas ===\n");
    for (int i = 0; i < max_linhas; i++) {
        printf("%s\n", matriz[i]); // Imprime cada linha da matriz
        free(matriz[i]); // Libera a memória da linha após a impressão
    }

    // Liberar memória alocada para o array de ponteiros
    free(matriz);
    printf("=====\n");
}
```

Figura 12 - Lista Antenas de Forma Formatada

## 2.4. Local Nefasto

### 2.4.1. Cabeçalho (LocalNefasto.h)

Declarou-se aqui as funções para calcular e listar locais nefastos. (Figura 13)

```
#pragma once
#ifndef LOCALNEFASTO_H
#define LOCALNEFASTO_H

#include "Dados.h" // Já define ED_LocaisNefastos

// Função principal
void calcularLocaisNefastos(ED* ed, ED_LocaisNefastos* locaisNefastos, int max_linhas, int max_colunas);

// Função auxiliar (não declarada no .h)
void listarLocaisNefastos(const ED_LocaisNefastos* locais);

void listarLocaisNefastosFormatado(const ED_LocaisNefastos* locais, int max_linhas, int max_colunas);

#endif // LOCALNEFASTO_H
```

Figura 13 - Cabeçalho LocalNefasto(.h)

### 2.4.2. Implementação (LocalNefasto.cpp):

Nesta classe foram implementados os métodos declarados anteriormente como:

**localExiste:** Confirma se um local nefasto já está presente na lista. (Figura 14).

```
// Verifica se um local nefasto já existe na lista
static int localExiste(ED_LocaisNefastos* locais, int linha, int coluna) {
    LocalNefasto* atual = locais->cabeca;
    while (atual != NULL) {
        if (atual->linha == linha && atual->coluna == coluna) return 1;
        atual = atual->prox;
    }
    return 0;
}
```

Figura 14 - Local Nefasto Existe

**adicionaLocalNefasto:** Adiciona um novo local nefasto à lista, evitando duplicações. (Figura 15)

```

// Adiciona um local nefasto à lista
static void adicionarLocalNefasto(ED_LocaisNefastos* locais, int linha, int coluna) {
    if (localExiste(locais, linha, coluna)) return;

    LocalNefasto* novo = (LocalNefasto*)malloc(sizeof(LocalNefasto));
    if (novo == NULL) {
        perror("Erro ao alocar memória para LocalNefasto");
        exit(EXIT_FAILURE);
    }

    novo->linha = linha;
    novo->coluna = coluna;
    novo->prox = locais->cabeca;
    locais->cabeca = novo;
}

```

*Figura 15 - Adiciona Local Nefasto*

### **calcularLocaisNefastos:**

Agrupa antenas por frequência usando uma estrutura de dados temporária.

Para cada par de antenas da mesma frequência, calcula possíveis locais nefastos.

Garante que as posições calculadas estejam dentro dos limites da matriz.  
(Figura 16)

```

void calcularLocaisNefastos(ED* ed, ED_LocaisNefastos* locaisNefastos, int max_linhas, int max_colunas) {
    if (ed == NULL || locaisNefastos == NULL) return; // Verificação de segurança para evitar erros

    Antena* atual = ed->cabeca;
    ED* mapaFrequencias[256] = { 0 }; // Array para armazenar listas de antenas por frequência (ASCII)

    // Passo 1: Agrupar antenas por frequência
    while (atual != NULL) {
        char freq = atual->frequencia;
        if (mapaFrequencias[freq] == NULL) {
            mapaFrequencias[freq] = criarED(); // Cria uma nova lista para armazenar antenas dessa frequência
        }
        insereAntena(mapaFrequencias[freq], freq, atual->linha, atual->coluna); // Insere a antena na lista correta
        atual = atual->prox;
    }

    // Passo 2: Calcular locais nefastos para cada par de antenas de mesma frequência
    for (int freq = 0; freq < 256; freq++) {
        if (mapaFrequencias[freq] == NULL) continue; // Se não houver antenas dessa frequência, pula para a próxima

        Antena* a1 = mapaFrequencias[freq]->cabeca;
        while (a1 != NULL) {
            Antena* a2 = a1->prox; // Evita pares duplicados (a1, a2) e (a2, a1)
            while (a2 != NULL) {
                // Diferença entre as coordenadas das duas antenas
                int dx = a2->coluna - a1->coluna;
                int dy = a2->linha - a1->linha;

                // Calcular as duas possíveis posições com efeito nefasto
                int L1_linha = a1->linha + 2 * dy;
                int L1_coluna = a1->coluna + 2 * dx;

                int L2_linha = a2->linha - 2 * dy;
                int L2_coluna = a2->coluna - 2 * dx;

                // Validação: Garante que os locais nefastos calculados estão dentro dos limites da matriz
                if (L1_linha >= 0 && L1_linha < max_linhas && L1_coluna >= 0 && L1_coluna < max_colunas) {
                    adicionarLocalNefasto(locaisNefastos, L1_linha, L1_coluna);
                }

                if (L2_linha >= 0 && L2_linha < max_linhas && L2_coluna >= 0 && L2_coluna < max_colunas) {
                    adicionarLocalNefasto(locaisNefastos, L2_linha, L2_coluna);
                }

                a2 = a2->prox; // Avança para o próximo par de antenas
            }
            a1 = a1->prox; // Avança para a próxima antena na lista
        }
    }

    // Passo 3: Liberar memória das listas temporárias usadas para agrupamento de antenas
    for (int i = 0; i < 256; i++) {
        if (mapaFrequencias[i] != NULL) {
            destruirED(mapaFrequencias[i]);
        }
    }
}

```

*Figura 16 - Calcula Locais Nefastos*

Listagem:

**listarLocaisNefastos:** Exibe coordenadas dos locais nefastos diretamente na consola. (Figura 17)

```

void listarLocaisNefastos(const ED_LocaisNefastos* locais) {
    // Verifica se a lista de locais nefastos está vazia
    if (locais == NULL || locais->cabeca == NULL) {
        printf("Nenhum local nefasto encontrado.\n");
        return;
    }

    LocalNefasto* atual = locais->cabeca; // Ponteiro para percorrer a lista ligada
    printf("=== Locais Nefastos ===\n");

    // Percorre a lista e imprime cada local nefasto
    while (atual != NULL) {
        printf("Local nefasto em (%d, %d)\n", atual->linha, atual->coluna);
        atual = atual->prox; // Avança para o próximo nó da lista
    }

    printf("=====\n");
}

```

Figura 17 - Listar Locais Nefastos

**listarLocaisNefastosFormatado:** Cria uma matriz visual com caracteres # para locais nefastos e "." para áreas livres. (Figura 18)

```

void listarLocaisNefastosFormatado(const ED_LocaisNefastos* locais, int max_linhas, int max_colunas) {
    // Verifica se a estrutura de locais é válida e se contém locais
    if (locais == NULL || locais->cabeca == NULL) {
        printf("Nenhum local nefasto encontrado.\n");
        return; // Se não houver locais nefastos, exibe mensagem e retorna
    }

    // Aloca dinamicamente a matriz de caracteres para armazenar a representação do mapa
    char** matriz = (char**)malloc(max_linhas * sizeof(char*)); // Aloca o vetor de ponteiros para as linhas
    for (int i = 0; i < max_linhas; i++) {
        matriz[i] = (char*)malloc(max_colunas * sizeof(char)); // Aloca cada linha da matriz
        for (int j = 0; j < max_colunas; j++) {
            matriz[i][j] = '.'; // Inicializa cada posição da matriz com o caractere '.'
        }
    }

    // Preenche a matriz com os locais nefastos representados por '#'
    LocalNefasto* atual = locais->cabeca; // Acessa o primeiro local da lista
    while (atual != NULL) {
        // Marca o local nefasto na posição correspondente da matriz
        matriz[atual->linha][atual->coluna] = '#';
        atual = atual->prox; // Avança para o próximo local na lista
    }

    // Imprime a matriz formatada, representando o mapa de locais nefastos
    printf("=== Mapa de Locais Nefastos ===\n");
    for (int i = 0; i < max_linhas; i++) {
        for (int j = 0; j < max_colunas; j++) {
            printf("%c", matriz[i][j]); // Imprime cada caractere da matriz
        }
        printf("\n"); // Quebra de linha ao final de cada linha da matriz
    }
    printf("=====\n");

    // Libera a memória alocada para a matriz
    for (int i = 0; i < max_linhas; i++) {
        free(matriz[i]); // Liberta cada linha da matriz
    }
    free(matriz); // Liberta o vetor de ponteiros para as linhas
}

```

Figura 18 - Lista Locais Nefastos Formatado

## 2.5. Projeto\_EDA.cpp (main.c)

Por fim criou-se o arquivo principal (main) do projeto que conta com as seguintes funcionalidades:

### 2.5.1. `exibirMenu` (Menu Interativo)

Exibe opções para listar antenas, remover/inserir antenas, calcular e listar locais nefastos.

Utiliza um loop para manter o programa em execução até a escolha da opção de sair.

```
void exibirMenu() {  
    printf("\nMenu:\n");  
    printf("1 - Listar antenas\n");  
    printf("2 - Listar antenas formatadas\n");  
    printf("3 - Remover uma antena\n");  
    printf("4 - Inserir uma antena\n");  
    printf("5 - Calcular locais nefastos\n");  
    printf("6 - Listar locais nefastos\n");  
    printf("7 - Listar locais nefastos formatados\n");  
    printf("8 - Sair\n");  
    printf("Escolha uma opcao: ");  
}
```

*Figura 19 - Menu Interativo*

### 2.5.2. `main` (Função principal).

Gestão de Antenas:

Carrega as antenas a partir de um arquivo (Antenas.txt) no início.

Oferece opções para listar, inserir e remover antenas.

Usa funções específicas para operações com listas ligadas.

Cálculo e Listagem de Locais Nefastos:

Calcula os locais nefastos com base nas antenas cadastradas.

Oferece listagem simples e formatada dos locais nefastos.

Gestão de Memória:

Cria e destrói a estrutura de dados principal (ED) ao iniciar e encerrar o programa.

Liberta memória alocada ao sair.

```

int main() {
    int opcao;
    ED* ed = criarED();
    ED_LocaisNefastos locaisNefastos = { NULL };
    int max_linhas, max_colunas;

    // Carregar antenas do ficheiro antes de exibir o menu
    carregarAntenasDeFicheiro(ed, nomeArquivo, &max_linhas, &max_colunas);

    do {
        exibirMenu();
        scanf_s("%d", &opcao);

        switch (opcao) {
            case 1:
                listarAntenas(ed);
                break;
            case 2:
                listarAntenasFormatado(ed, max_linhas, max_colunas);
                break;
            case 3:
                listarAntenas(ed);
                removerAntenaManual(ed);
                break;
            case 4:
                listarAntenas(ed);
                inserirAntenaManual(ed);
                break;
            case 5:
                calcularLocaisNefastos(ed, &locaisNefastos, max_linhas, max_colunas);
                printf("Calculo feito com sucesso.\n");
                break;
            case 6:
                listarLocaisNefastos(&locaisNefastos);
                break;
            case 7:
                listarLocaisNefastosFormatado(&locaisNefastos, max_linhas, max_colunas);
                break;
            case 8:
                printf("A Sair...\n");
                break;
            default:
                printf("Opcao invalida! Tente novamente.\n");
        }
    } while (opcao != 8);

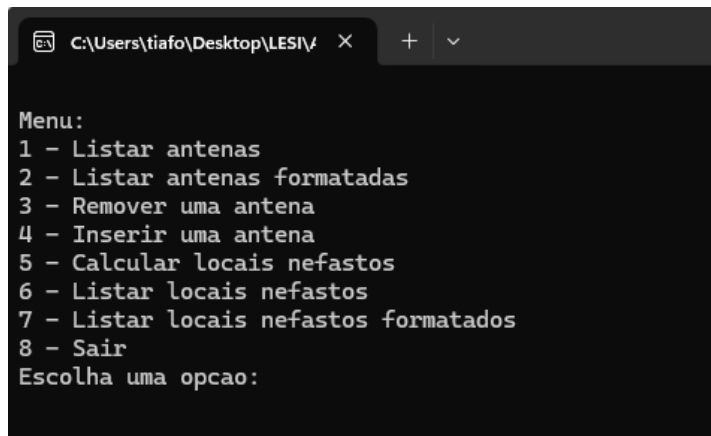
    destruirED(ed);
    return 0;
}

```

Figura 20 - Função principal (Main)

## 2.6. Testes ao programa.

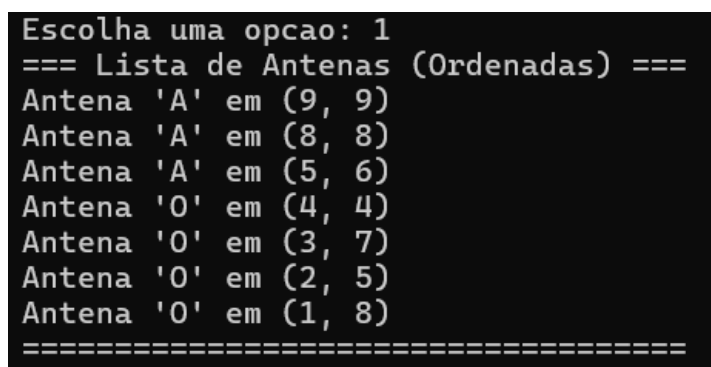
### 2.6.1. Menu



```
Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao:
```

Figura 21 - Menu

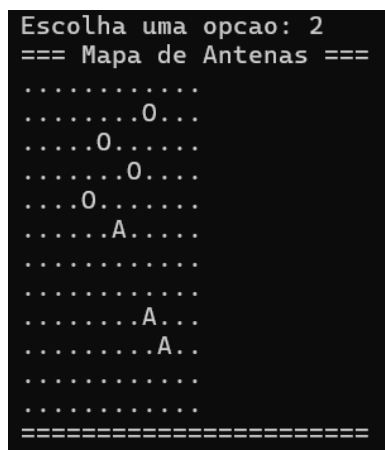
### 2.6.2. Listar Antenas



```
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 22 - Lista de Antenas Consola

### 2.6.3. Listar Antenas Formatado



```
Escolha uma opcao: 2
=== Mapa de Antenas ===
.....
.....0...
.....0....
.....0....
.....0....
.....A....
.....
.....
.....A...
.....A..
.....
.....
=====
```

Figura 23 - Lista Antenas Formatado Consola



#### 2.6.4. Inserir Antena

```
Insira a frequencia da antena (caractere) a acrescentar: A
Insira a linha da antena: 5
Insira a coluna da antena: 5

Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 5)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 24 - Inserir Antena Consola

#### 2.6.5. Remover Antena

```
=====
Insira a frequencia da antena (caractere) a remover: A
Insira a linha da antena: 5
Insira a coluna da antena: 5

Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 25 - Remover Antena Consola

### 2.6.6. Listar Locais Nefastos

```
Escolha uma opcao: 6
=== Locais Nefastos ===
Local nefasto em (2, 10)
Local nefasto em (5, 1)
Local nefasto em (0, 6)
Local nefasto em (6, 3)
Local nefasto em (4, 9)
Local nefasto em (7, 0)
Local nefasto em (5, 6)
Local nefasto em (0, 11)
Local nefasto em (3, 2)
Local nefasto em (7, 7)
Local nefasto em (10, 10)
Local nefasto em (1, 3)
Local nefasto em (2, 4)
Local nefasto em (11, 10)
=====
```

Figura 26 - Lista Locais Nefastos Consola

### 2.6.7. Listar Locais Nefastos Formatado

```
=== Mapa de Locais Nefastos ===
.....#.....#
...#.....
....#.....#.
..#.....
.....#..
.#.....#.
...#.....
#.....#.
.....
.....
.....#.
.....#.
=====
```

Figura 27 - Lista Locais Nefastos Formatada Consola

### 3. Conclusão

A conclusão desta primeira fase evidencia a implementação bem-sucedida dos conceitos fundamentais de estruturas de dados dinâmicas, especificamente através da utilização de listas ligadas para a gestão de antenas e locais nefastos. Foram implementadas funções robustas para a inserção, remoção e listagem de antenas, além da leitura e interpretação de ficheiros com matrizes de caracteres, o que permitiu a identificação correta das posições ocupadas e vazias.

Adicionalmente, foi desenvolvido um mecanismo para calcular os locais nefastos, baseado na análise de pares de antenas com frequências iguais, garantindo a aplicação correta das regras de alinhamento e distância. A integração destas funcionalidades num menu interativo, aliado ao uso de técnicas seguras de leitura e gestão de memória, demonstra a eficácia da abordagem adotada para resolver um problema de dimensão média.

Esta fase não só consolidou os conhecimentos teóricos adquiridos, mas também permitiu a aplicação prática de conceitos de modularização, documentação e verificação de segurança, servindo de base sólida para o desenvolvimento da próxima etapa do projeto, onde serão exploradas técnicas mais avançadas com a utilização de grafos.

## 4. Bibliografia

Link Repositório GIT : [https://github.com/Tiago20452/Projeto\\_EDA\\_20452.git](https://github.com/Tiago20452/Projeto_EDA_20452.git)